

LocalClass

Децентрализованная P2P-система обмена
сообщениями и файлами в локальной сети

ВЕРСИЯ 1.0

Учебный проект уровня Middle / Hard
Целевая конфигурация: 15-20 узлов в одной сессии
Платформа: Windows (.exe), Python
Дата документа: 8 сентября 2026

1	Назначение и границы проекта
2	Ключевые архитектурные решения
3	Топология и устройство узла
4	Идентичность и доверие
5	Discovery и работа с сетевыми интерфейсами
6	Диагностика сети
7	Транспорт, framing и версионирование протокола
8	Событийная модель: device_seq, Lamport, vector clock
9	Распространение событий: gossip и anti-entropy
10	Порядок событий и tombstone
11	Хранилище и схема данных
12	Передача файлов
13	Сессии, QR и роли
14	Журналирование
15	Тестируемость и fault injection
16	Спецификация протокола (каркас следующего документа)
17	Технологический стек и структура кода
18	План спринтов
19	Критерии готовности
20	Реестр известных граблей
21	Открытые вопросы

1. Назначение и границы проекта

LocalClass — desktop-приложение для обмена сообщениями, ссылками и файлами между компьютерами учебной группы внутри одной локальной сети, без обязательного подключения к интернету и без центрального сервера.

1.1. Функциональные возможности

- автоматическое обнаружение компьютеров в сети;
- P2P mesh-коммуникация между равноправными узлами;
- текстовый чат с каналами;
- обмен ссылками с отдельной лентой;
- передача файлов произвольного размера с возобновлением;
- синхронизация пропущенных событий после отключения;
- работа при частичной связности сети;
- диагностика сетевых проблем с понятными причинами;
- роли Student / Teacher;
- подключение к сессии по коду или QR;
- подробное журналирование двух уровней;
- сборка в Windows .exe с установщиком.

1.2. Что в первую версию не входит

Явно исключено, чтобы проект оставался реализуемым:

- распределённый консенсус, leader election, Byzantine fault tolerance;
- relay файлов через промежуточные узлы;
- полноценное шифрование трафика (TLS откладывается);
- аудио- и видеосвязь;
- синхронизация файловых деревьев целиком;
- интернет-сервер, облачные аккаунты, внешняя авторизация;
- swarm-раздача файлов (получил кусок — раздал дальше).

1.3. Учебная ценность

Основная сложность проекта лежит не в чате и не в интерфейсе, а в сетевой части: discovery при меняющихся IP, множественные сетевые интерфейсы, реконнект, синхронизация после потери связи и корректная работа при частичной связности. Именно эти задачи превращают проект из «файлообменника по Wi-Fi» в небольшую распределённую систему.

2. Ключевые архитектурные решения

Раздел фиксирует принятые решения вместе с обоснованием. Он же служит защитой от повторного обсуждения одних и тех же вопросов на середине разработки.

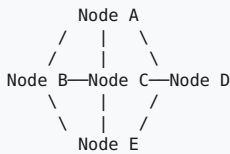
Решение	Обоснование
Нет Host, полный mesh	Схема «главный компьютер + backup + выборы нового хоста» требует распределённого консенсуса: split-brain, два хоста одновременно, гонки при обнаружении падения. Для 15 узлов полная mesh (105 соединений) дешевле и отказоустойчива по конструкции.
Нет глобального счётчика событий	Глобальную нумерацию некому вести без сервера. Вместо неё — счётчик у каждого узла и вектор версий.
Gossip без TTL	Распространение завершается за счёт дедупликации по event_id . Hop limit создаёт тихую потерю сообщений в длинных цепочках.
Anti-entropy обязателен	Gossip «по факту получения» не покрывает потерю отдельного пакета при живом соединении. Периодический обмен векторами закрывает дыру.
Порядок по Lamport, не по часам	Часы школьных ноутбуков расходятся на минуты. Сортировка по wall-clock перемешивает чат.
device_id = отпечаток публичного ключа	UUID в файле подделывается блокнотом. Ed25519 стоит того же объёма кода и даёт настоящую идентичность.
Файлы — отдельное соединение	Чанки по общему WebSocket блокируют чат на всё время передачи.
Файлы не идут через gossip	Relay гигабайтов через чужой ноутбук по общей Wi-Fi-среде неприемлем. Нет прямого пути — нет передачи, и это честно показывается.
Критическое состояние — в SQLite	Счётчики и состояние передач в RAM ломают resume после перезапуска и порядок событий после падения.
Диагностика — часть MVP	«Ничего не находится» — самый частый сценарий. Без диагностики отладка превращается в гадание.

главный принцип

IP — временный адрес транспорта. device_id — постоянная криптографическая личность. session_id — идентификатор группы. События — основной объект синхронизации. Ни одна из этих сущностей не выводится из IP-адреса.

3. Топология и устройство узла

3.1. Полносвязная mesh



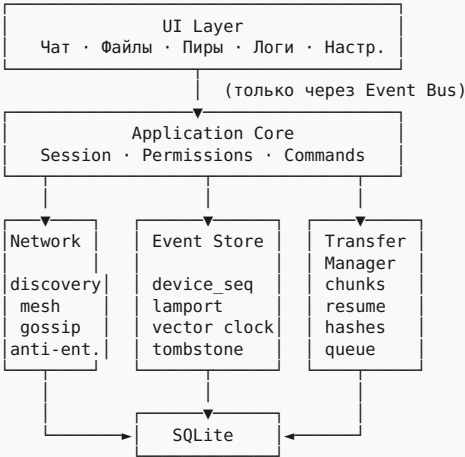
Максимум соединений при 15 узлах: $15 \times 14 / 2 = 105$

Каждый компьютер — равноправный Node. Выключение любого узла не останавливает систему: остальные продолжают работать без переконфигурации.

3.2. Что хранит каждый узел

- криптографическую идентичность (приватный и публичный ключ);
- локальную базу SQLite;
- полный журнал событий своей сессии;
- сообщения, ссылки, tombstone;
- состояние файловых передач;
- таблицу известных пиров и их отпечатков;
- персистентные счётчики `device_seq` и `lamport`.

3.3. Внутренняя структура узла

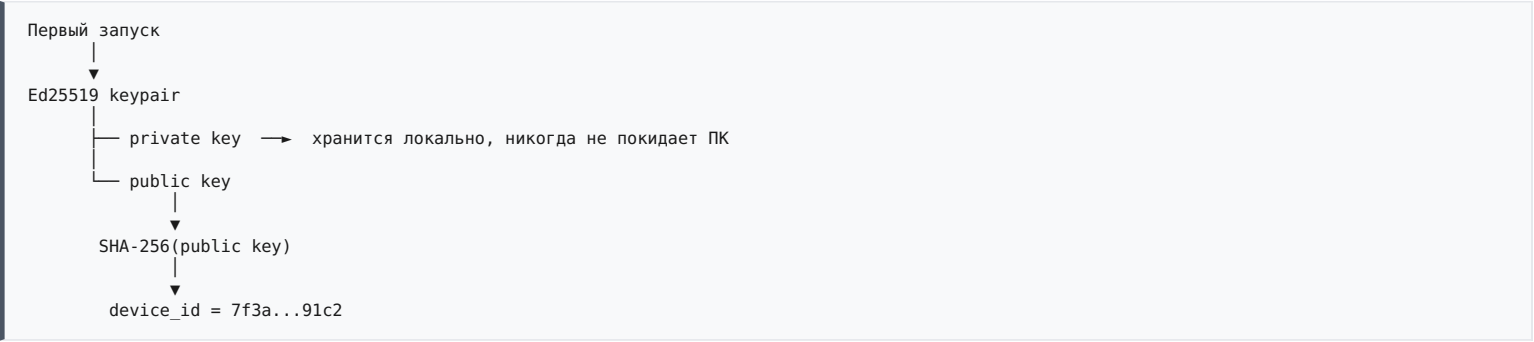


ПРАВИЛО РАЗРАБОТКИ

GUI не обращается к сети напрямую и не пишет в базу. Любое действие пользователя превращается в команду ядра, любое изменение состояния приходит в UI событием. Это позволяет запускать узел вообще без интерфейса — что критично для автоматических тестов.

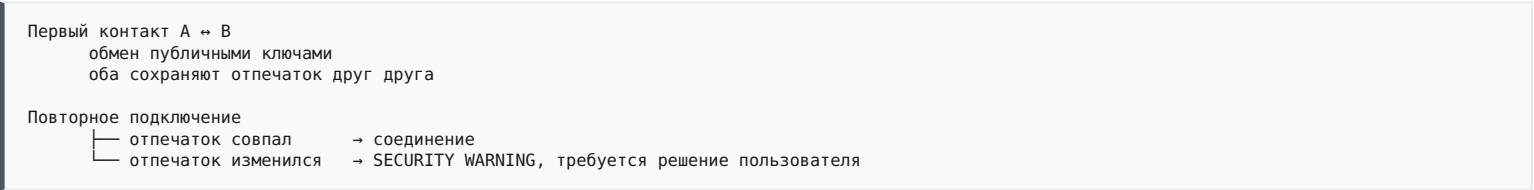
4. Идентичность и доверие

4.1. Криптографическая личность



Смена IP-адреса не создаёт нового устройства: 192.168.1.10 → 192.168.1.27 для системы остаётся тем же device_id.

4.2. TOFU — Trust On First Use



ВАЖНО НЕ ПЕРЕПУТАТЬ

Криптографическая идентичность узла — это **не** доверие к пользователю. Она гарантирует «это тот же компьютер, что и вчера», но ничего не говорит о том, кто за ним сидит и не модифицирован ли клиент.

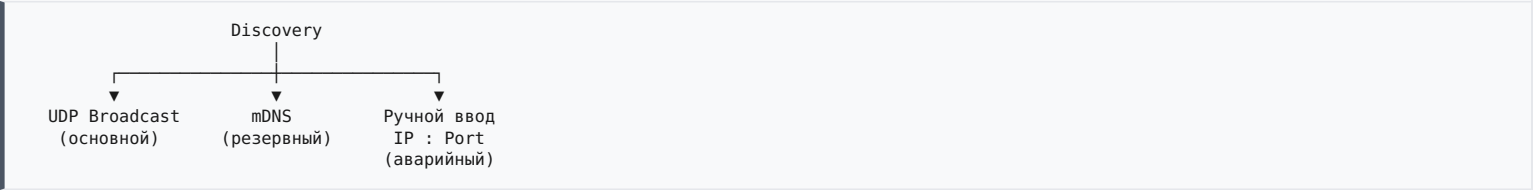
4.3. Роли как cooperative security

Роли Student / Teacher в базовой версии проверяются клиентом, а клиент находится в руках ученика. Любой, кто откроет исходники, сможет объявить себя Teacher.

Это приемлемо для учебного класса, но должно быть записано прямо, а не подразумеваться. Настоящая авторизация — отдельное расширение: команды преподавателя подписываются его приватным ключом, публичный ключ распространяется через QR при создании сессии, каждый узел проверяет подпись самостоятельно.

5. Discovery и работа с сетевыми интерфейсами

5.1. Три механизма обнаружения



Два автоматических механизма нужны потому, что сети ведут себя по-разному: где-то режется multicast и проходит broadcast, где-то наоборот. Ручное подключение — последний рубеж, когда не работает ничего.

5.2. Содержимое discovery-пакета

```
{
  "protocol_version": 1,
  "device_id": "7f3a...91c2",
  "public_key": "...",
  "session_id": "8f4c...",
  "port": 45821,
  "addresses": ["192.168.1.25", "192.168.0.12"]
}
```

Собственный `device_id` всегда отбрасывается — иначе узел найдёт сам себя. Фильтрация именно по `device_id`, а не по IP: при запуске нескольких узлов на одном компьютере IP у них совпадает.

5.3. Множественные сетевые интерфейсы

Интерфейсы типичного ноутбука:		
Wi-Fi	192.168.1.25	← вероятно рабочий
Ethernet	192.168.0.12	← возможно рабочий
VPN	10.8.0.5	← почти наверняка бесполезен здесь
VirtualBox	192.168.56.1	← мусор
Docker	172.17.0.1	← мусор
Loopback	127.0.0.1	← только для локальных тестов

ОШИБКА, КОТОРУЮ ДЕЛАЮТ ВСЕ

Взять первый найденный IP и считать его рабочим. Список интерфейсов не отсортирован по полезности, и на машине с VirtualBox первым вполне может оказаться 192.168.56.1. Рабочий адрес определяется только фактически установленным соединением, а не эвристикой по имени интерфейса.

В настройках пользователь должен видеть список интерфейсов и иметь возможность включить или отключить каждый вручную.

5.4. Несколько узлов на одном компьютере

Обязательный режим разработки. Каждый экземпляр получает свои `device_id`, порт, базу и хранилище:

```
node --device A --port 5001 --data ./run/A
node --device B --port 5002 --data ./run/B
node --device C --port 5003 --data ./run/C
```

ТЕХНИЧЕСКАЯ МИНА

Все экземпляры захотят слушать один UDP-порт для broadcast. Потребуется `SO_REUSEADDR` (а на Windows — учёт специфики приёма собственных broadcast-пакетов). Заложить это в Sprint 1, иначе тестовый режим не заработает вообще.

6. Диагностика сети

Диагностика входит в MVP, а не добавляется в конце. Причина простая: сценарий «запустили, никого не видно» будет самым частым, и без диагностики отладка сводится к перезагрузке роутера наугад.

6.1. Экран диагностики

Диагностика сети	
Wi-Fi подключён	✓
IPv4-адрес получен	✓
UDP broadcast отправлен	✓
mDNS доступен	✓
TCP-порт слушается	✓
Обнаружено узлов	12
Доступно узлов	10
Недоступно	2
Возможные причины:	
• изоляция клиентов на точке доступа • блокировка Firewall • разные подсети • маршрутизация через VPN	
[Подключиться вручную] [Повторить]	

6.2. Разница между «обнаружен» и «доступен»

Узел может быть виден в discovery, но недостижим по TCP — типичная картина при client isolation на точке доступа. Эти два состояния должны различаться в UI явно.

ПРАВИЛО

Никаких тихих ошибок. Если сообщение не может дойти до конкретного участника, интерфейс обязан это показать. Худший класс багов в такой системе — тот, при котором пользователь уверен, что сообщение отправлено, а его никто не получил.

7. Транспорт, framing и версионирование

7.1. Обязательные поля любого пакета

```
protocol_version
message_type
request_id
sender_id
payload
```

7.2. Length-prefix framing

length 4 B big-endian	payload N bytes
--------------------------	-----------------

ЗАПРЕЩЕНО

Строить фрейминг на `\n` или других строковых разделителях. TCP — поток байтов без границ сообщений; разделитель ломается на первом же сообщении, содержащем перевод строки, а в чате они будут постоянно.

7.3. Совместимость версий

Обновить 15 ноутбуков одновременно невозможно, поэтому разные версии узлов будут работать в одной сети. Требования:

- неизвестный **тип** сообщения — игнорируется, узел не падает;
- неизвестное **поле** в известном типе — игнорируется;
- несовместимая версия протокола — `PROTOCOL_VERSION_MISMATCH` и понятное сообщение пользователю: «Версия LocalClass на этом компьютере устарела».

Тесты совместимости пишутся с первого дня:

```
v1 ↔ v1    ✓
v1 ↔ v2    ✓ (деградация функций, но соединение работает)
v2 ↔ v2    ✓
```

7.4. Кодировка

JSON, UTF-8. Для 15 узлов производительность MessagePack не даёт выигрыша, а JSON несравнимо проще отлаживать. Ограничения фиксируются явно: без `NaN` и `Infinity`, целые числа не превышают 2^{53} .

8. Событийная модель

Все изменения состояния системы представляются событиями. Событие — единственный объект, который передаётся между узлами (кроме содержимого файлов).

8.1. Структура события

```
{
  "version": 1,
  "event_id": "e7c1a4...",      случайный уникальный ID
  "device_id": "7f3a...",       автор события
  "device_seq": 42,             счётчик автора
  "lamport": 187,               логическое время
  "timestamp": 1757359200,       только для показа человеку
  "type": "MESSAGE_CREATED",
  "payload": { "channel": "general", "text": "Привет" }
}
```

8.2. Три счётчика и их роли

Механизм	За что отвечает
device_seq	Наличие или отсутствие события. Позволяет понять, какие события пропущены.
vector clock	Синхронизация. Это не отдельная сущность, а представление {device_id → последний известный device_seq}.
lamport	Логический порядок отображения. Сортировка по (lamport, device_id, event_id).
timestamp	Только вывод времени в интерфейсе. В логике не участвует никогда.

8.3. Правила работы со счётчиками

```
При создании локального события:
  lamport += 1
  device_seq += 1

При получении чужого события:
  lamport = max(lamport, remote_lamport) + 1
```

ДВЕ ОШИБКИ, КОТОРЫЕ ЧИНЯТСЯ ТОЛЬКО СБРОСОМ БАЗЫ У ВСЕХ

1. Счётчики не персистятся. После перезапуска узел начинает с нуля, переиспользует номера, и синхронизация решает, что события уже есть. Оба счётчика хранятся в SQLite.

2. Инкремент счётчика и запись события — разные транзакции. При падении между ними в последовательности образуется дыра, и супс зависает, бесконечно ожидая несуществующее событие.

```
BEGIN TRANSACTION
  UPDATE counters SET device_seq = device_seq + 1
  INSERT INTO events (...)
COMMIT
```

8.4. Каталог типов событий

MESSAGE_CREATED	MESSAGE_DELETED	
LINK_CREATED		
FILE_OFFERED	FILE_COMPLETED	FILE_FAILED
USER_JOINED	USER_LEFT	
PEER_CONNECTED	PEER_DISCONNECTED	
CHANNEL_CREATED	CHANNEL_CLOSED	
SESSION_CREATED	SESSION_CLOSED	
PERMISSIONS_UPDATED		

9. Распространение событий: gossip и anti-entropy

9.1. Проблема частичной связности

A ↔ B ✓

B ↔ C ✓

A ↔ C ✗

изоляция клиентов или Firewall на одной машине

Полный mesh молча предполагает, что все видят всех. В реальной школьной сети это неверно. Событие от А должно дойти до С через В.

9.2. Правила gossip

Узел получил событие:

1. проверил event_id в базе

2. уже известно → игнорировать, дальше не передавать

3. новое → сохранить

4. передать всем подходящим пирам

5. не отправлять обратно тому, от кого получил

ПОЧЕМУ НЕТ TTL

Распространение и так завершается за счёт дедупликации по event_id : узел, уже видевший событие, дальше его не шлёт. Hop limit не нужен, но при этом активно вреден — в цепочке А—В—С—D событие с TTL = 2 не дойдёт до D, и никто об этом не узнает. Ограничиваются размер события, частота и размер очередей — но не число хопов.

9.3. Почему одного gossip недостаточно

Сценарий тихой потери:

А создал сообщение

А → В пакет потерялся в эфире

соединение А-В при этом осталось живым

реконнекта нет → синхронизация не запускается

сообщение потеряно навсегда

9.4. Anti-entropy

Периодическая фоновая сверка, работающая независимо от реконнектов:

каждые 10–30 секунд

▼

выбрать случайного доступного пира

▼

обменяться vector clock

▼

вычислить расхождение

▼

передать недостающие события в обе стороны

Стоит около десяти строк кода и превращает доставку из «обычно доходит» в «доходит гарантированно».

ГАРАНТИЯ СИСТЕМЫ

Gossip + anti-entropy дают **eventual delivery**, а не мгновенную доставку. Если между А и С периодически существует путь — прямой или через промежуточные узлы — событие в конечном итоге дойдёт. Если узлы физически изолированы, доставка невозможна, и UI обязан показывать это состояние.

9.5. Ограничение потока

Роли — cooperative security, значит модифицированный клиент может залить сто тысяч событий в секунду, и они по gossip осядут в базах всего класса. Это защита не столько от злоумышленника, сколько от бага в собственном коде.

- лимит событий в секунду на device_id (ориентир: 100);
- лимит размера события (ориентир: 256 КБ);
- ограниченная очередь на приём, при переполнении события отбрасываются с записью в лог;
- пир, стабильно превышающий лимиты, временно отключается;
- после отключения anti-entropy восстанавливает корректные события.

Все пороги конфигурируемы.

10. Порядок событий и tombstone

10.1. Порядок доставки не гарантирован

Прямое следствие gossip: события приходят разными путями и обгоняют друг друга. Совершенно нормальная последовательность:

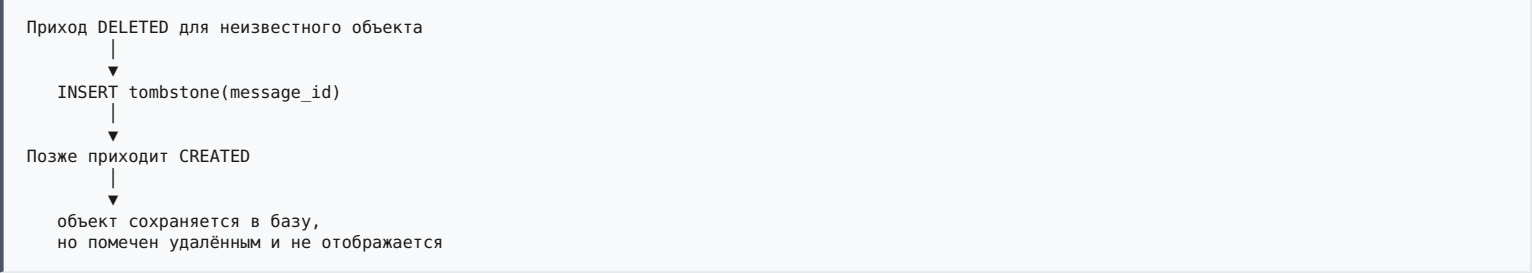
```
MESSAGE_DELETED (message_id = 42)
  ↓
MESSAGE_CREATED (message_id = 42)
```

Наивная реализация либо уронит обработчик, либо проигнорирует удаление — и сообщение «воскреснет».

10.2. Правило применения событий

ПРАВИЛО

Событие применяется независимо от порядка поступления. Удаление хранится как **tombstone**: пришло `DELETED` для неизвестного `message_id` — сохранить `tombstone`; когда придёт `CREATED`, сообщение не показывается. `Tombstone` живёт до конца срока хранения, а не удаляется сразу после применения.



То же правило распространяется на удаление файлов, закрытие каналов и любые другие операции отмены.

11. Хранилище и схема данных

11.1. Расположение

%APPDATA%/LocalClass/		
config/		
settings.json		
identity/		
device.key	приватный ключ, только локально	
database/		
local.db	SQLite	
storage/		
incoming/	карантин незавершённых передач	
files/	принятые файлы	
logs/		
2026-09-08.log		

11.2. Основные таблицы

counters	device_id, device_seq, lamport ← персистентно, обновляется в транзакции с events
events	event_id PK, device_id, device_seq, lamport, timestamp, type, payload, session_id UNIQUE (device_id, device_seq, session_id)
tombstones	object_id, object_type, deleted_by, lamport
peers	device_id PK, public_key_fingerprint, last_addresses, last_seen, trust_state
sessions	session_id PK, name, code, created_at, duration, teacher_public_key, closed_at
transfers	transfer_id PK, file_id, peer_id, direction, status, filename, size, chunk_size, chunk_count, received_chunks, full_sha256, updated_at
chunk_hashes	transfer_id, chunk_index, hash

11.3. Область действия vector clock

ПРОБЛЕМА РОСТА

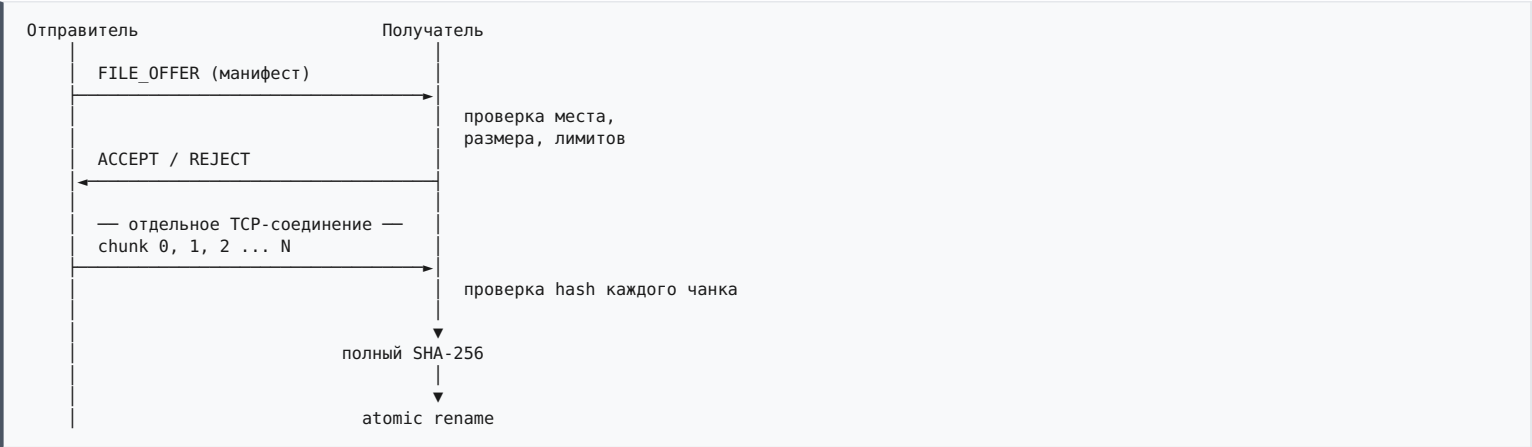
За учебный год через приложение пройдут сотни `device_id`. Без ограничения вектор превратится в словарь на триста записей, который гоняется при каждой сверке.

Решение: вектор скоупится **сессией**, а не устройством. Плюс срок хранения — события закрытой сессии удаляются через настраиваемый период (по умолчанию 30 дней).

Retention применяется к событиям, сообщениям, tombstone, векторам, записям о передачах и временным файлам.

12. Передача файлов

12.1. Общая схема



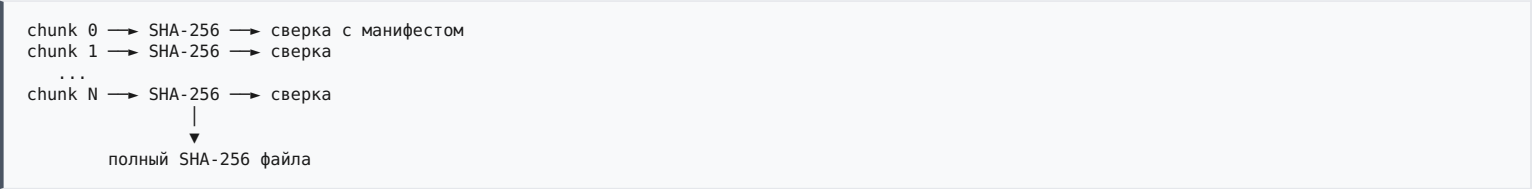
12.2. FILE_OFFER — манифест до передачи

```
file_id
filename
size
chunk_size
chunk_count
full_sha256
chunk_hashes[] ← список хешей всех чанков
```

ЗАЧЕМ МАНИФЕСТ ЗАРАНЕЕ

Без списка хешей получатель не может проверить чанк независимо, и resume превращается в «поверь отправителю». С манифестом получатель заранее знает структуру файла, проверяет место на диске до начала приёма и может запросить повторно ровно один битый чанк.

12.3. Двухуровневая проверка целостности



Хеш от 1,4 ГБ считается заметное время, поэтому повреждение обнаруживается на уровне чанка сразу, а не в конце передачи. Повторно запрашивается только чанк 47, а не файл целиком.

12.4. Resume

Состояние передачи хранится в SQLite, а не в памяти. Иначе возобновление работает при обрыве связи, но не при перезапуске приложения — а перезапускать будут постоянно.

```
Возобновление после:
потери Wi-Fi
закрытия приложения
перезагрузки Windows

Передаются только отсутствующие или повреждённые чанки.
```

12.5. Физика Wi-Fi

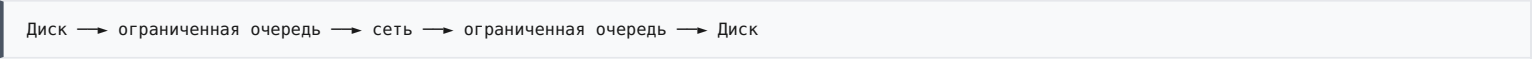
ОГРАНИЧЕНИЕ СРЕДЫ

Одна точка доступа — общая полудуплексная среда, а не 15 независимых каналов. Реальный агрегат — десятки мегабит на всех. Отправка файла 1,4 ГБ четырнадцати ученикам создаёт **19,6 ГБ** трафика в эфир и парализует класс примерно на сорок минут, включая чат.

- Требования:
- жёсткий лимит одновременных исходящих передач (ориентир: 3, не 14);
 - очередь вместо параллельного доступа;
 - предупреждение в UI до начала: «Примерно 19,6 ГБ трафика, передача займёт значительное время»;

- честная оценка времени по фактической скорости, а не только полоса прогресса.

12.6. Backpressure



Скорость отправителя адаптируется к получателю. Без ограничения буфера медленный получатель раздувает память отправителя до OOM.

12.7. Подтверждение чанков

Поштучный ACK на гигабайтном файле убивает скорость: на каждый чанк приходится round-trip. Используется **оконное** подтверждение — отправитель держит в полёте N неподтверждённых чанков.

12.8. Безопасность приёма

Имя файла из сети — недоверенный ввод.

Опасно: ../../Windows/System32/file.exe
 C:\Windows\...
 /etc/passwd

Правило: санитизируется ПУТЬ, а не алфавит имени.

Unicode сохраняется: Домашка №3.zip
Недопустимо: ____3.zip

Порядок приёма:

1. санитизировать путь
2. проверить размер против лимита
3. проверить свободное место на диске
4. создать временный файл в incoming/
5. принять чанки, проверяя хеши
6. проверить полный SHA-256
7. atomic rename в files/

Автоматический запуск принятого файла отсутствует всегда.

12.9. Лимиты

MAX_FILE_SIZE
MAX_CONCURRENT_OUTGOING
MAX_CONCURRENT_INCOMING
MAX_CHUNK_SIZE
MAX_STORAGE_SIZE

ориентир: 3

12.10. Файлы и частичная связность

События: A → B → C gossip разрешён
Файлы: A — X — C передача невозможна

Relay файлов в первую версию не входит. Если прямого пути нет, пользователю показывается явная причина, а не бесконечная попытка соединения.

13. Сессии, QR и роли

13.1. Модель сессии

session_id	настоящий идентификатор
session_name	«Python Evening»
session_code	K7F2-X9 — для ввода человеком
duration	длительность, а не абсолютный expires_at
created_at	
teacher_public_key	

ДВЕ ТОНКОСТИ

- Коды не уникальны. Два преподавателя в соседних классах могут сгенерировать одинаковый K7F2-X9 . Код служит только для поиска и ручного ввода; подключение всегда матчится по session_id .
- Срок жизни задаётся длительностью, а не меткой времени окончания — по той же причине, по которой порядок сообщений не строится на часах компьютера.

13.2. Содержимое QR-кода

```
{
  "version": 1,
  "session_id": "8f4c...",
  "session_code": "K7F2-X9",
  "teacher_public_key": "...",
  "addresses": [ { "host": "192.168.1.10", "port": 45821 } ]
}
```

QR не заменяет discovery, а даёт дополнительный путь подключения — в том числе когда автоматическое обнаружение не работает.

13.3. Роли

Роль	Возможности
Student	чат в разрешённых каналах, отправка файлов и ссылок, скачивание, просмотр участников
Teacher	дополнительно: создание и закрытие каналов, удаление сообщений и файлов, блокировка пользователей, объявления, управление доступом

Ограничение из раздела 4.3 остаётся в силе: в базовой версии это кооперативная функция, а не защита.

14. Журналирование

Два независимых потока, которые нельзя смешивать.

Тип	Содержание	Аудитория
Application log	DEBUG, INFO, WARNING, ERROR, CRITICAL — техническая сторона: TCP connection reset, WebSocket reconnect, Discovery timeout	разработчик, режим Advanced
Event log	MESSAGE_CREATED, FILE_COMPLETED, PEER_CONNECTED — что произошло в приложении	пользователь и преподаватель

Ошибка Connection reset by peer не обязательно является событием приложения. И наоборот: FILE_COMPLETED — событие, которое дополнительно порождает обычную запись в технический лог.

14.1. Фильтры в интерфейсе

[Все] [INFO] [WARNING] [ERROR] [DEBUG]

[Сеть] [Файлы] [Чат] [Сессия] [Безопасность]

Дополнительно: по узлу, по пиру, по интервалу времени

15. Тестируемость и fault injection

ПРИНЦИП

Тестируемость закладывается в архитектуру с первого спринта, а не добавляется потом. Без неё команда будет отлаживаться, бегая между ноутбуками и выдёргивая Wi-Fi, — а воспроизвести баг синхронизации таким способом невозможно.

15.1. Множественный запуск

```
node --device A --port 5001 --data ./run/A
node --device B --port 5002 --data ./run/B
node --device C --port 5003 --data ./run/C
```

Каждый узел получает отдельные device_id, порт, базу и хранилище.
Узел должен запускаться без GUI — только ядро.

15.2. Внесение неисправностей

Доступно в debug-режиме:

packet loss	A → B : 20 %
latency	A → B : 300 ms
connection drop	разрыв конкретной пары
random disconnect	случайные обрывы
partition	искусственная изоляция A x C

15.3. Обязательные сценарии тестов

Сценарий	Что проверяется
Узел офлайн 5 минут, затем возврат	восстановление пропущенных событий, отсутствие дубликатов
Потеря пакетов 20 % при живом соединении	работа anti-entropy (gossip этот случай не покрывает)
Разрыв A X C при живых A—B—C	доставка событий через промежуточный узел
Перезапуск процесса в середине передачи	resume из SQLite, целостность чанков
Перезапуск узла с существующей базой	персистентность lamport и device_seq, отсутствие перемешивания истории
Приём события DELETED раньше CREATED	корректная работа tombstone
Узел v1 подключается к узлу v2	совместимость протокола, игнорирование неизвестных полей
Пир превышает лимит событий	rate limiting, отключение, последующее восстановление

16. Спецификация протокола — каркас следующего документа

Это ТЗ фиксирует архитектуру. Следующий документ — точная спецификация протокола: перечень пакетов, их поля и алгоритмы обработки. До неё писать сетевой код не следует, иначе четыре разработчика напишут четыре несовместимых сериализатора.

16.1. Перечень пакетов

Соединение	HELLO, HELLO_ACK, AUTH, PEER_LIST, PING, PONG, GOODBYE
Синхронизация	SYNC_REQUEST, SYNC_RESPONSE, ANTI_ENTROPY_PROBE
События	EVENT, EVENT_ACK
Файлы	FILE_OFFER, FILE_ACCEPT, FILE_REJECT, FILE_CHUNK, FILE_CHUNK_ACK, FILE_RESUME, FILE_FINISH, FILE_ABORT
Служебные	ERROR, PROTOCOL_VERSION_MISMATCH

16.2. Вопросы, которые обязана закрыть спецификация

Вопрос	Почему это важно
Пагинация SYNC_RESPONSE	Узел был офлайн неделю — расхождение в десятки тысяч событий. Одним пакетом их не отдать. Нужен явный признак «это не всё, запроси следующую порцию».
Одновременный запуск anti-entropy	Два узла выбрали друг друга в один момент. Должно получиться либо два независимых обмена, либо один — но выбрать надо явно, иначе поведение будет случайным.
Размер окна ACK для чанков	Поштучное подтверждение убивает скорость на больших файлах.
Поведение при частичном ответе	Соединение оборвалось в середине SYNC_RESPONSE — что считать полученным.
Точные границы значений	Максимальный размер пакета, кодировка, пределы чисел.

ОРГАНИЗАЦИОННОЕ ПРАВИЛО

Спецификацию протокола пишет **один человек** с ревью остальных. Протокол — то место, где коллективное творчество даёт худший результат, чем один связный автор.

17. Технологический стек и структура кода

17.1. Стек

Python 3.11+	
PySide6	GUI
asyncio	сетевое ядро
websockets	или чистый asyncio-транспорт
SQLite + SQLAlchemy	хранилище
cryptography	Ed25519
zeroconf	mDNS
PyInstaller	сборка .exe

ПРО FASTAPI

FastAPI нужен, если интерфейс веб-ный. С PySide6 он лишний: придётся мирить цикл uvicorn с циклом Qt через qasync или отдельный поток. Берём PySide6 + чистый asyncio.

17.2. Структура проекта

localclass/	
core/	
identity.py	ключи, device_id, T0FU
events.py	модель события, счётчики
store.py	SQLite, транзакции
sync.py	vector clock, anti-entropy
gossip.py	распространение, дедупликация
net/	
discovery.py	UDP broadcast, mDNS
interfaces.py	перечисление интерфейсов
transport.py	framing, соединения
protocol.py	пакеты, версии
diagnostics.py	проверки сети
files/	
manifest.py	FILE_OFFER, хеши
transfer.py	чанки, окно, backpressure
storage.py	карантин, санитизация, rename
ui/	
debug/	простой интерфейс со Sprint 2
app/	полноценный GUI
testing/	
faults.py	packet loss, latency, partition
harness.py	запуск N узлов в одном процессе
node.py	точка входа без GUI

17.3. Сборка

- PyInstaller собирает LocalClass.exe ;
- установщик предлагает ярлык, автозапуск и разрешение в Windows Firewall;
- при первом запуске Windows спросит про доступ в частных сетях — это ожидаемо;
- **неподписанный exe ловит SmartScreen** — предупредить группу заранее, иначе половина класса не сможет запустить приложение.

18. План спринтов

Семь этапов. Каждый даёт работающий результат, который можно показать.

Sprint 1 — Network Foundation

identity (Ed25519, device_id)
перечисление сетевых интерфейсов
UDP broadcast discovery
mDNS
список пиров
диагностика сети
ручное подключение IP:Port
запуск нескольких узлов на одном ПК
фильтрация собственного device_id

Результат: узлы видят друг друга и объясняют, почему соединение невозможно.

Sprint 2 — Mesh и Debug UI

TCP / WebSocket соединения
length-prefix framing
protocol_version
TOFU-обмен ключами
heartbeat, reconnect
модель событий
SQLite
базовый чат
простой отладочный интерфейс
два уровня логов

Результат: работающий P2P-чат между несколькими узлами.

Sprint 3 — Синхронизация и gossip

device_seq (персистентный, транзакционный)
vector clock
Lamport clock
event store
sync-протокол
anti-entropy
gossip и дедупликация
tombstone
rate limiting
работа при частичной связности
retention

Результат: отключили узел на пять минут, вернули — данные восстановились. Событие от А дошло до С через В при разорванной прямой связи.

ДИСЦИПЛИНА

Sprint 4 не начинается, пока Sprint 3 не проходит fault-injection тесты. Соблазн будет — файлы визуально эффектнее синхронизации. Но чинить event store, когда поверх него уже лежит передача файлов, вдвое дороже.

Sprint 4 — Файлы

FILE_OFFER с манифестом
чанкование
хеши чанков и полный SHA-256
resume из SQLite
карантин и atomic rename
санитизация путей
лимиты и очередь
backpressure, оконный ACK
отдельное соединение
прогресс и оценка времени

Результат: надёжная передача больших файлов без блокировки чата.

Sprint 5 — Сессии

session_id и session_code
QR-код
teacher_public_key
duration и retention
сохранение и восстановление сессии
подключение после перезапуска
привязка TOFU к сессии

Sprint 6A — Collaboration

роли Student / Teacher
каналы и права доступа
объявления
лента ссылок
личные сообщения

Sprint 6B — Product Polish

полноценный GUI
drag-and-drop
уведомления, системный трей
clipboard sharing, скриншоты
поиск по сообщениям и файлам
Markdown в чате
настройки
автозапуск, installer, EXE

Оценка

Для группы из 3-4 человек — примерно 5-7 месяцев при двух занятиях в неделю. Sprint 3 самый тяжёлый и самый ценный: если группа его вытянет, она разберётся в распределённых системах лучше большинства выпускников.

19. Критерии готовности

19.1. Функциональные

Система готова, если на 10-15 компьютерах можно последовательно выполнить:

1. установить EXE
 2. создать сессию
 3. подключиться по коду или QR
 4. автоматически обнаружить участников
 5. увидеть диагностику сети
 6. отправить сообщение
 7. получить его без обновления интерфейса
 8. отправить ссылку
 9. передать файл 1 ГБ и больше
 10. видеть прогресс и оценку времени
 11. пользоваться чатом во время передачи
 12. оборвать соединение
 13. восстановить его
 14. продолжить передачу файла с места обрыва
 15. восстановить пропущенные события
 16. работать при частичной связности
 17. выключить один узел без остановки остальных
 18. перезапустить узел и восстановить состояние
 19. увидеть все действия в логах
 20. получить внятную причину при проблемах Firewall / VPN / изоляции

19.2. Численные

Критерий	Требование
Синхронизация	после 5 минут офлайна узел восстанавливает пропущенные события менее чем за 10 секунд после возврата связи при штатной нагрузке
Передача файла	файл 1 ГБ между двумя узлами по стабильному Wi-Fi доходит без потери и повреждения чанков
Отзывчивость чата	во время передачи большого файла чат продолжает принимать и отображать сообщения без блокировки интерфейса
Устойчивость к потерям	при 20 % потери пакетов все события в итоге доходят до всех связанных узлов
Отсутствие дубликатов	после серии реконнектов ни одно сообщение не отображается дважды

20. Реестр известных граблей

Список составлен из ошибок, которые в проектах такого типа допускаются практически всегда. Каждая проверяется на ревью соответствующего спринта.

Спринт	Грабли	Последствие
1	Взять первый IP из списка интерфейсов	приложение слушает адрес VirtualBox и не видит никого
1	Забыть <code>SO_REUSEADDR</code>	несколько узлов на одном ПК не запускаются, тестовый режим не работает
1	Фильтровать себя по IP, а не по <code>device_id</code>	узел видит сам себя при локальном запуске
2	Фрейминг по <code>\n</code>	любое сообщение с переводом строки ломает поток
2	Падение на неизвестном поле	невозможно обновлять класс постепенно
3	Счётчики только в памяти	после перезапуска история перемешивается; читится только сбросом баз у всех
3	Инкремент и запись в разных транзакциях	дыра в последовательности, супс зависает навсегда
3	TTL в gossip	тихая недоставка в длинных цепочках
3	Синхронизация только по реконнекту	потерянный пакет при живом соединении теряется навсегда
3	Сортировка по <code>timestamp</code>	сообщения прыгают из-за расхождения часов
3	DELETED раньше CREATED без tombstone	удалённое сообщение воскресает
3	Вектор без привязки к сессии	неограниченный рост за учебный год
4	Состояние передачи в памяти	resume не переживает перезапуск приложения
4	Хеш только полного файла	повреждение видно в конце, resume не проверяем
4	Чанки по общему соединению	чат встаёт на время передачи
4	Отсутствие backpressure	OOM отправителя на медленном получателе
4	Поштучный ACK	катастрофическая скорость на больших файлах
4	Санитизация алфавита вместо пути	<code>Домашка №3.zip</code> превращается в <code>_____3.zip</code>
4	Веерная отправка всем сразу	19,6 ГБ в эфир, класс парализован
5	Матчинг сессии по коду	соседние группы сливаются при совпадении кода
5	<code>expires_at</code> вместо длительности	сессия закрывается не вовремя из-за часов
6	Роли как заявленная защита	ложное чувство безопасности

21. Открытые вопросы

Решения, сознательно отложенные до спецификации протокола или до следующих версий. Они не блокируют начало работы, но должны быть закрыты явно, а не забыты.

Вопрос	Когда решать
Пагинация SYNC_RESPONSE и поведение при обрыве в середине	спецификация протокола, до Sprint 3
Разрешение одновременного anti-entropy между двумя узлами	спецификация протокола
Размер окна ACK и адаптация к скорости канала	Sprint 4
Политика при исчерпании MAX_STORAGE_SIZE	Sprint 4
Поведение при смене отпечатка пира (авто-отказ или вопрос пользователю)	Sprint 5
TLS поверх транспорта	после v1.0
Подписанные команды преподавателя	после v1.0
Swarm-раздача файлов	после v1.0, если понадобится

Итог

АРХИТЕКТУРНАЯ БАЗА V1.0

Нет Host. Нет leader election. Нет глобального счётчика событий. Нет TTL в gossip. Нет зависимости порядка от системных часов. Нет передачи файлов через промежуточные узлы. Нет критического состояния только в оперативной памяти.

Discovery отвечает за поиск узлов. Mesh — за соединения. События вместе с gossip и anti-entropy — за гарантированную в итоге доставку и синхронизацию. Отдельный файловый транспорт — за большие файлы.

Документ достаточно конкретен, чтобы распределить работу между несколькими разработчиками и начать реализацию, не оставляя критические архитектурные решения «на потом». Следующий шаг — спецификация протокола (раздел 16).